Streamlit 웹 대시보드 개발 보고서

1. 개요 및 특징

Streamlit(스트림릿)은 파이썬 언어만으로 데이터 분석 결과를 대화형 웹 대시보드로 생성하고 공유할 수 있게 해주는 오픈소스 라이브러리입니다. HTML, CSS, JavaScript와 같은 프론트엔드 지식 없이도 머신러닝 및 데이터 과학 프로젝트를 위한 웹 앱을 빠르게 구축할 수 있다는 장점이 있습니다.

- **주요 특징**:
  - 파이썬 코드와 Streamlit 코드가 별도의 구분 없이 혼합되어 사용됩니다.
  - 데이터나 소스 코드가 변경될 때마다 전체 파이썬 스크립트를 위에서 아래로 다시 실행하는 **Data Flow** 구조를 가집니다.

---

2. 개발 환경 구축 및 실행

- **가상환경**: `conda`를 이용한 `wordfreq` 환경
- **주요 라이브러리**: `numpy`, `pandas`, `matplotlib` (데이터 분석), `konlpy` (한글 처리), `wordcloud` (시각화)
- **IDE**: Visual Studio Code (VS Code)

  `pip install streamlit`

  `# 설치 확인 및 데모 실행`
  `streamlit hello`
- **실행 방법**: 터미널에서 `streamlit run 파일명.py` 명령어를 입력합니다.

---

3. 핵심 API 가이드

3.1 텍스트 및 상태 출력

Streamlit은 다양한 텍스트 출력 함수를 제공하며, 출력 함수의 반환 객체를 통해 내용을 수정할 수 있습니다.

함수명	용도	특징
`st.title()`	메인 제목 출력	가장 큰 글자 크기
`st.header()`	섹션 헤더	title보다 작은 크기

`st.write()`	범용 데이터 출력	데이터 타입에 맞춰 자동 시각화
`st.info()`	정보 메시지 (Blue)	사용자 안내용
`st.success()`	성공 메시지 (Green)	작업 완료 알림
`st.warning()`	경고 메시지 (Yellow)	주의 사항 알림
`st.error()`	오류 메시지 (Red)	에러 발생 알림

> **Magic Command**: 특정 출력 함수를 쓰지 않아도 변수나 데이터를 코드에 적으면 자동으로 `st.write()`가 적용되어 화면에 출력됩니다.

3.2 입력 위젯 (Input Widgets)

사용자로부터 데이터를 입력받아 상호작용을 가능하게 합니다.

- **선택 도구**: `st.button()` (클릭 여부), `st.checkbox()` (체크 여부), `st.radio()` (단일 선택), `st.selectbox()` (드롭다운), `st.multiselect()` (다중 선택).
- **값 입력**: `st.slider()` (범위 조절), `st.text_input()` (문자열 입력), `st.number_input()` (숫자 입력).
- **사이드바**: `st.sidebar.함수명()`을 사용하여 화면 왼쪽에 입력 컨트롤을 배치할 수 있습니다.

3.3 폼(Form) 처리

여러 입력을 한 번에 제출하려면 `st.form`을 사용합니다.

- `with st.form(key="my_form"):` 블록 안에 위젯을 작성합니다.
- `st.form_submit_button()`이 눌렸을 때만 데이터를 일괄 처리하여 성능을 최적화합니다.

---

4. 데이터 시각화 및 최적화

4.1 시각화 라이브러리 연동

Matplotlib 등 외부 라이브러리로 생성한 차트를 화면에 표시할 수 있습니다.

Python

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.barh(...) # 차트 생성
st.pyplot(fig) # Streamlit 화면에 출력
```

4.2 성능 최적화 (Caching)

매번 코드를 재실행하는 특성상, 데이터 로딩이나 복잡한 연산은 캐싱이 필수입니다.

- `@st.cache_data`: 데이터프레임 로드나 함수 계산 결과를 저장하여 재실행 시 속도를 높입니다.

---

5. 실무 활용도가 높은 추가 기능

강의 자료 외에 실제 프로젝트에서 자주 활용되는 필수 기능들입니다.

5.1 레이아웃 제어 (Layouts)

화면 공간을 효율적으로 분할하기 위해 사용합니다.

- `st.columns()`: 화면을 가로로 분할하여 여러 요소를 병렬 배치합니다.
- `st.tabs()`: 탭 인터페이스를 제공하여 콘텐츠를 분류합니다.
- `st.expander()`: 클릭 시 내용이 펼쳐지는 접이식 컨테이너를 만듭니다.

5.2 데이터 디스플레이 및 미디어 (업데이트)

다양한 형태의 데이터와 텍스트를 용도에 맞게 시각화합니다.

- `st.dataframe()`: 인터랙티브한 표(정렬, 검색 가능)를 출력하며, `use_container_width=True` 설정으로 화면 너비에 맞출 수 있습니다.
- `st.metric()`: 지표와 변화량(Delta)을 카드 형태로 표시합니다.
- `st.file_uploader()`: 사용자가 자신의 파일을 업로드하여 분석에 활용하게 합니다.
- `st.code()`: 프로그래밍 소스 코드를 문법 하이라이팅과 함께 출력하며, 'Copy' 버튼을 지원합니다.
- `st.caption()`: 본문보다 작은 글씨로 주석, 출처, 설명 등을 달 때 사용합니다.
- `st.latex()`: LaTeX 문법을 활용하여 복잡한 수학 공식이나 수식을 깔끔하게 렌더링합니다.

5.3 세션 상태 관리 (Session State)

- `st.session_state`: 페이지가 재실행되어도 변수 값을 유지해야 할 때 사용합니다. 위젯 간의 데이터 공유나 로그인 상태 유지 등에 필수적입니다.

5.4 진행 상태 표시

사용자 경험(UX)을 개선하기 위해 작업의 진행 상황을 시각적으로 알립니다.

- `st.spinner()`: 연산 중임을 알리는 로딩 애니메이션을 일시적으로 보여줍니다.

- `st.progress()`: 0%에서 100%까지 진행률 바를 표시하여 작업 완료 시점을 예상하게 합니다.

5.5 개발자 도구 및 기타 레이아웃

개발 과정의 효율성을 높이고 섹션을 명확히 구분하는 기능입니다.

- `st.echo()`: `with` 블록 안의 코드를 화면에 텍스트로 보여줌과 동시에, 해당 코드가 실제로 실행된 결과물도 나란히 출력합니다. 튜토리얼이나 코드 설명 시 유용합니다.
- `st.header(divider=True)`: 헤더 아래에 구분선을 추가하여 콘텐츠의 구획을 명확히 나눕니다.

---

6. 결론

Streamlit은 데이터 분석가와 개발자가 가장 직관적으로 결과물을 공유할 수 있는 도구입니다. 기본적인 텍스트 및 위젯 API를 익히고, 캐싱과 레이아웃 기능을 적절히 조합한다면 강력한 인터랙티브 웹 대시보드를 구축할 수 있습니다.